Spotify Dataset Analysis and Popularity Prediction Using Deep Learning

CS171: Introduction to Machine Learning

Jonathan Wu
Department of Computer Science

December 18, 2025

Contents

1 Introduction **2**

2 Data Preprocessing **2**
- 2.1 Data Imbalance . . . 2
- 2.2 Feature Engineering . . . 2
- 2.3 Principal Component Analysis . . . 2
- 2.4 Train, Validation, and Test Sets . . . 3

3 Baseline Model **3**
- 3.1 Baseline Model Results . . . 4
- 3.2 Model Evaluation . . . 4

4 Hyperparameter Tuning and Results **5**
- 4.1 Baseline Hyperparameters . . . 5
  - 4.1.1 Hidden Layers . . . 6
  - 4.1.2 Hidden Layer Size . . . 6
  - 4.1.3 Dropout . . . 6
  - 4.1.4 Activation Functions . . . 6
  - 4.1.5 Kernel Initializers . . . 6
- 4.2 Loss Function . . . 7
- 4.3 Optimizers . . . 7
  - 4.3.1 Learning Rates . . . 7
- 4.4 Learning Rate Schedule . . . 7

5 Hyperparameter Summary **12**

6 Results **12**

7 Other Models **13**
- 7.1 SVM . . . 13
- 7.2 Random Forest . . . 14

8 Future Work **14**

1 Introduction

The objective of this project is to understand the fundamentals of various concepts within machine learning by applying practical models to predict the popularity of a song based on the information provided in a Spotify dataset.

The following sections will provide the data preprocessing steps, deep learning architecture choices, model training, and evaluation of the model's predictions. The report will provide an analysis of the model's predictive capability and offer recommendations for future work.

2 Data Preprocessing

2.1 Data Imbalance

When examining the dataset, we found that there it is an imbalanced dataset that represents more non-popular songs (0) than popular songs (1).

```
Class ratios:
popular
0    0.784575
1    0.215425
```

This is to be expected as a majority of songs in reality are not popular. A small portion of songs will become popular, therefore, we do not want to balance our dataset to have an equal amount of popular and non-popular songs. Artificially balancing the dataset by undersampling non-popular tracks or oversampling popular tracks would distort the true data distribution and reduce the model's ability to generalize to real-world scenarios. Instead, class weighting is used during model training to penalize misclassification of the minority (popular) class more heavily. This approach preserves the natural distribution of the data while encouraging the model to learn meaningful decision boundaries for identifying popular songs.

2.2 Feature Engineering

We preprocessed certain columns in our dataset to ensure that it is in a more effective form to train our model. For example, track_duration_ms and artist_followers were transformed using log to utilize smaller numbers while retaining proportional relationships to one another. This allows our model to train faster. We extracted only the year from the album_release_date to help prevent overfitting. We also used binary encoding for explicit instead of TRUE or FALSE.

Our final lists of features composes of the following:

- track_number
- track_duration_ms
- explicit
- artist_popularity
- artist_followers
- album_total_tracks
- album_release_year
- album_type
- artist_genres

2.3 Principal Component Analysis

Principal Component Analysis (PCA) was applied after imputation and feature scaling to reduce multicollinearity among numerical features. PCA was primarily evaluated for neural network and

margin-based models, while tree-based models were trained on the original feature space.

The explained variance plot (Figure 2) indicates that approximately 3-4 principal components capture 80-90% of the total variance. However, empirical evaluation showed that PCA-based models underperformed compared to models trained on the original features, particularly in recall and PR-AUC. This suggests that PCA removed informative but low-variance signals that are critical for identifying rare popular tracks.

```
    ===== No PCA =====
Accuracy: 0.7949886104783599
Precision: 0.8421052631578947
Recall: 0.05653710247349823
F1: 0.10596026490066225
ROC-AUC: 0.8046131186308616
PR-AUC: 0.49172837818425275


===== With PCA (90% Variance) =====
Accuracy: 0.7851176917236142
Precision: 0.0
Recall: 0.0
F1: 0.0
ROC-AUC: 0.7612944344581064
PR-AUC: 0.4122280394733036
```

As a result, PCA was not used in the final neural network models. Instead, the original engineered features were retained to preserve class-discriminative information.

2.4 Train, Validation, and Test Sets

Train Sample %	Validation Sample %	Test Sample %
80	10	10
70	20	10

Table 1: Model Hyperparameters

We tested a baseline model with the splits from Table 1 and found a 80-10-10 split for the train-validation-test set was more accurate and provided a higher PR-AUC. This is likely because the model has more data to learn from with 80% compared to ony 70%, allowing for a better result.

3 Baseline Model

We implemented a baseline multi-layer perceptron model in Python using TensorFlow and Keras. The hyperparameters found in Table 2 were used for the baseline model:

Hyperparameter	Value
Hidden Layers	2
Hidden Layer Size	10
Dropout	0
Activation Function	ReLU
Kernel Initializer	glorot-uniform
Output Activation	Sigmoid
Optimizer	Stochastic Gradient Descent
Loss Function	Binary Cross Entropy

Table 2: Baseline Model Hyperparameters

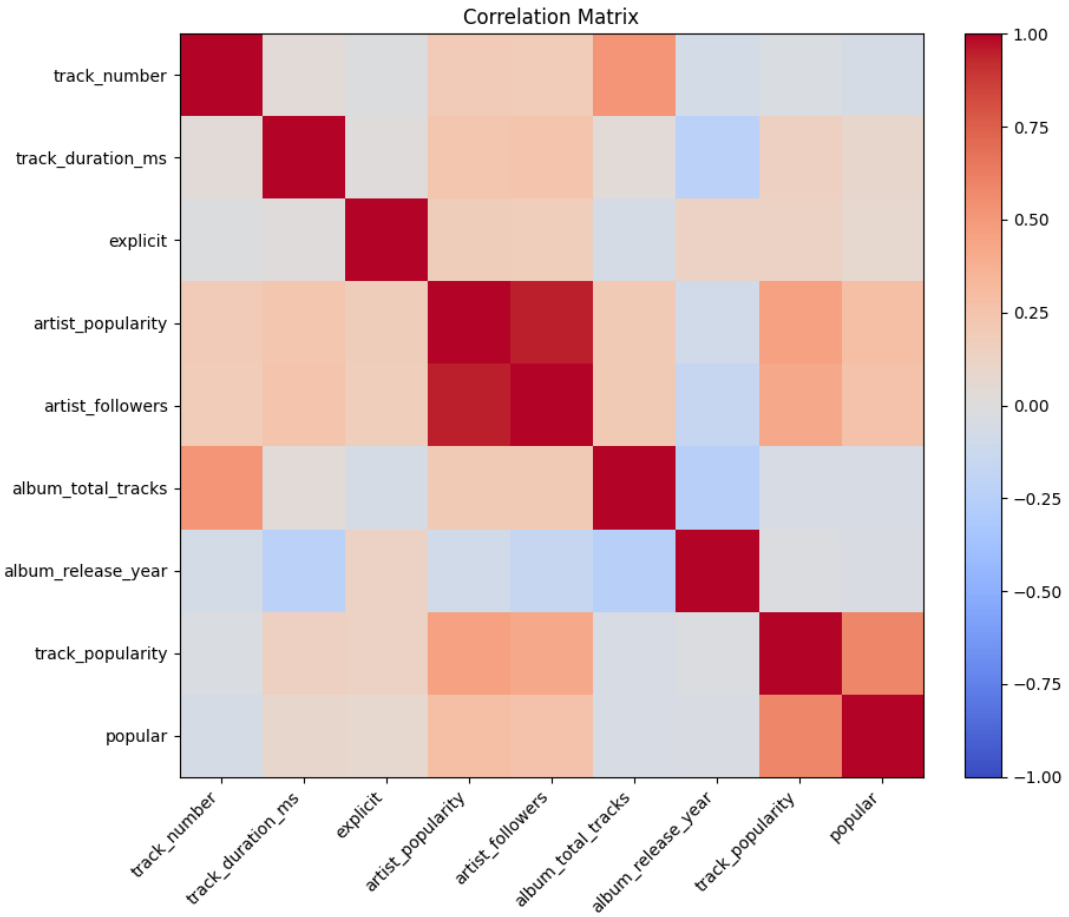


Figure 1: Correlation matrix

This correlation matrix describes the correlation between the different features provided in the dataset. Red represents a high correlation that we seen between artist_popularity and artist_followers.

Likewise, there is a high correlation between track_popularity and popular. Track_popularity was transformed from a scale of 0-99 to a binary classification of popular or not by considering if the song was in the top 20%, therefore, track_popularity will not be used as a feature to prevent label leakage.

3.1 Baseline Model Results

Log from running the baseline model:

```
Layers=2, Units=10, Dropout=0.0, Activation=relu, Init=glorot_uniform
28/28 ------------------- 0s 1ms/step
Val Acc=0.789, Prec=0.750, Recall=0.032, F1=0.061, ROC-AUC=0.795, PR-AUC=0.458
```

3.2 Model Evaluation

We evaluated the model's performance looking at four different metrics: precision, recall, ROC-AUC, and PR-AUC. Accuracy to determine the overall correctness of our model, precision to avoid false positives, and recall to catch rare popular tracks. Because we are working with an imbalanced dataset, we want to focus on having the highest PR-AUC.

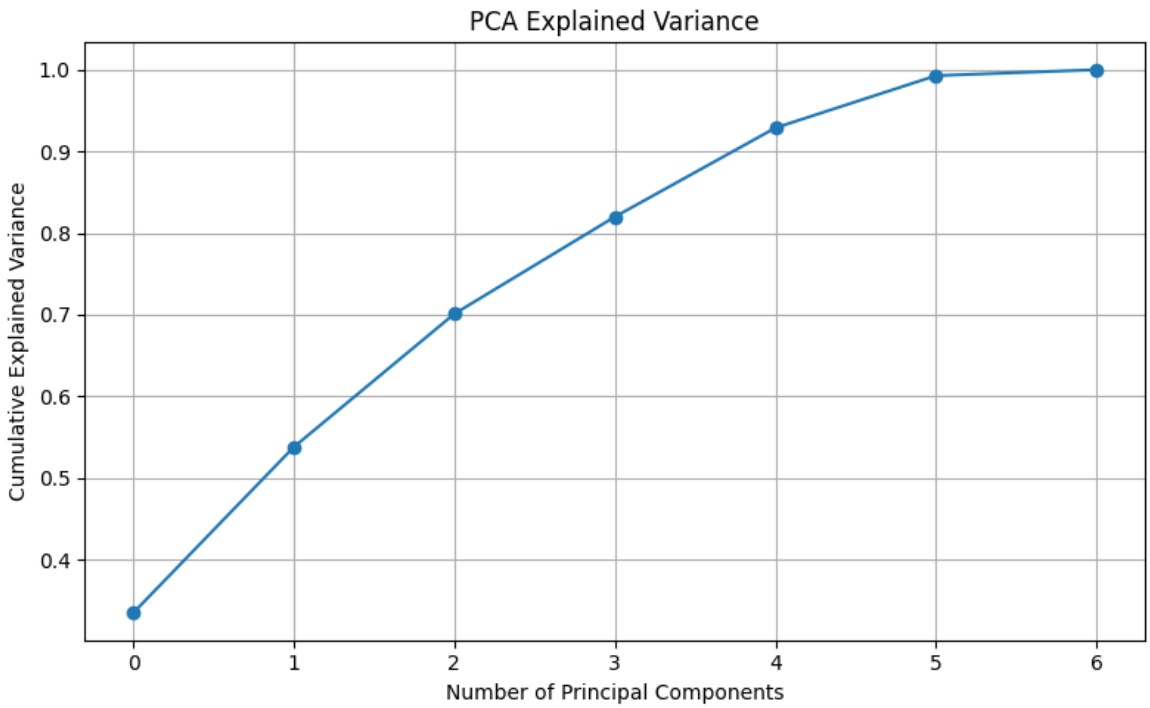


Figure 2: PCA Graph
Based on the PCA graph, the number of principal components that should be used is between 3 and 4 to capture 80%-90% of the variance.

4 Hyperparameter Tuning and Results

4.1 Baseline Hyperparameters

See Table 3 for all hyperparameters tested. Below is the result of our best model:

Hyperparameter	Values Tested	Best Result
Hidden Layers	1, 2, 3, 4, 5, 6	4
Hidden Layer Size	5, 10, 20, 40, 500	20
Dropout	0, 0.2, 0.5	0
Activation Function	tanh, ReLU, Elu	ReLU
Kernel Initializer	glorot-uniform, glorot-normal, uniform, normal	glorot-normal
Output Activiation	Sigmoid	Sigmoid

Table 3: Model Hyperparameters Tested
By testing 1080 different combinations of these hyperparameter values, we found that the best result was a model with 4 hidden layers, 20 nodes in each hidden layer, 0 dropout, ReLU activiation function, and glorot-normal kernel initializer.

```
==============================
BEST MODEL (by PR-AUC)
Best PR-AUC: 0.5232469549450459
Best Configuration: {'hidden_layers': 4, 'hidden_units': 20, 'dropout': 0.0,
'activation': 'relu', 'initializer': 'glorot_normal'}
==============================
```

4.1.1 Hidden Layers

We found the best result was 4 hidden layers. 1 to 3 hidden layers were too little to understand the dataset's complexity, whereas PR-AUC suffered with 5 to 6 layers due to overfitting.

4.1.2 Hidden Layer Size

We found the best size for each hidden layer was 20. The model has enough nodes to understand the dataset without creating too much noise seen with a size of 40 or 500. We can see the model is underfit in a model with only 5 layers that does not have enough complexity (Figure 3). Distinct overfitting can be seen in Figure 4 when 500 layers are used.

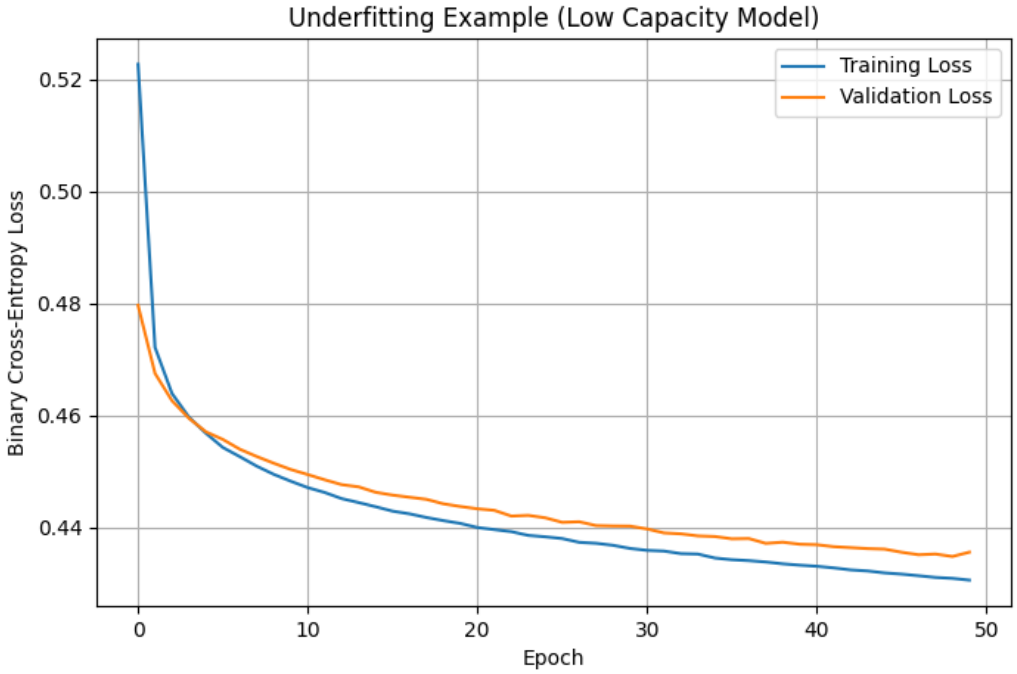


Figure 3: Underfit Model with 5 Layers
This graph demonstrates the model underfitting due to having too few nodes. The model is too simple to capture the complexity of the dataset and has a high loss.

4.1.3 Dropout

The validation loss of the model was stable and adding dropout will increase noise to the model and negatively impact the recall of rare classes. Because we have an imbalanced dataset, dropout would often suppress weak but important signals and lower PR-AUC.

4.1.4 Activation Functions

Three activation functions, tanh, ReLU, and Elu were used. ReLU performed the best as our input is already standardized. The sparse activations allows for stable gradients and works well with stochastic gradient descent. See Figure 5.

4.1.5 Kernel Initializers

Four different kernel initializers were tested: normal, uniform, glorot-uniform, and glorot-normal. Glorot-normal performed the best as it prevented gradients from vanishing or exploding which improved training stability and smoother loss curves. Normal initializer had more uncontrolled variance. Uniform had poor scaling and glorot-uniform was a bit noiser for deeper nets. See Figure 6.

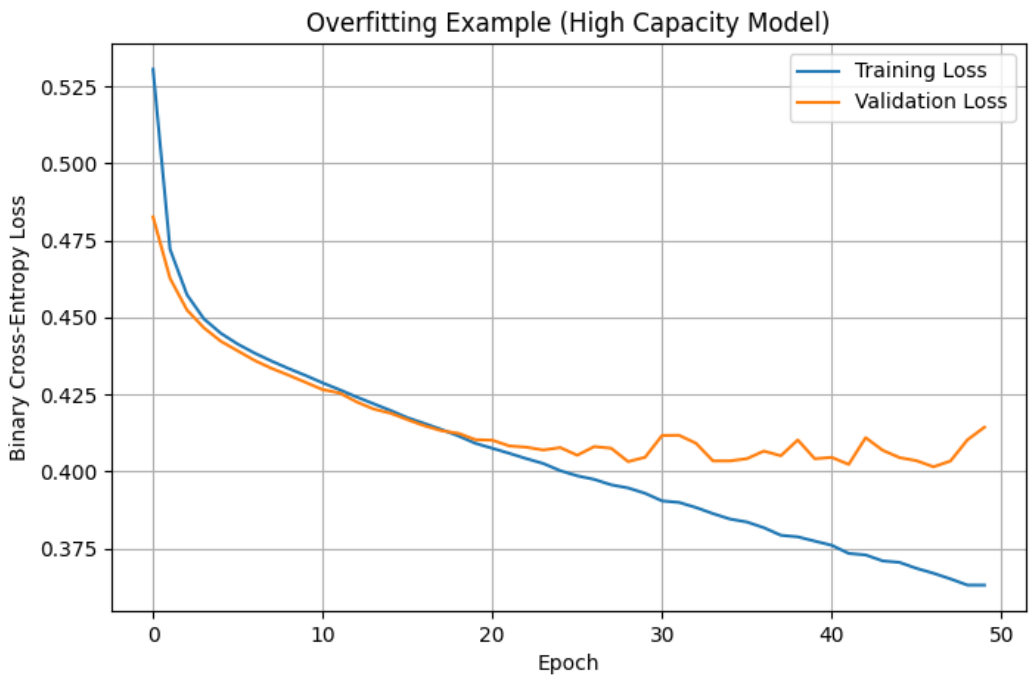


Figure 4: Overfitting Model with 500 Layers
This graph demonstrates the model overfitting due to having too many nodes in each hidden layer. This introduces too much noise and the model and causes overfitting to improve accuracy.

4.2 Loss Function

Since this is a binary classification problem, we tested three different loss functions: binary cross-entropy, hinge loss, and squared hinge. We found that binary cross-entropy and hinge loss performed the best as they had the same accuracy. However, hinge loss was better than binary cross-entropy because it had a higher PR-AUC score.

4.3 Optimizers

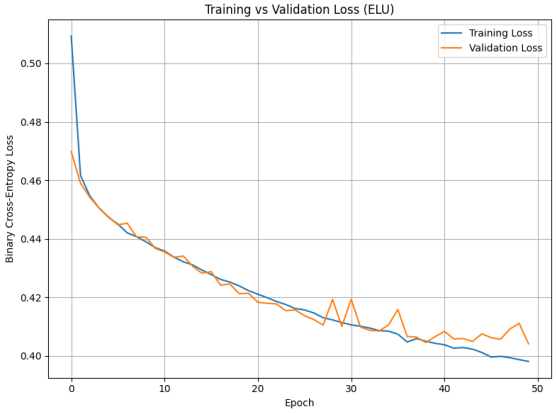
Three different optimizers were tested: stochastic gradient descent, Adam and RMSprop. Adam and RMSprop both overfit while stochastic gradient descent was found to perform the best. See Figure 7.

4.3.1 Learning Rates

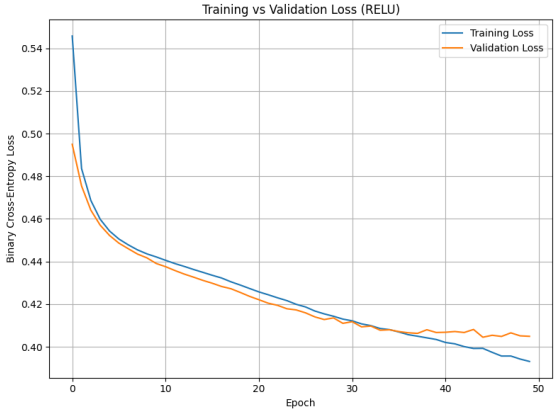
Since we found that Stochastic Gradient Descent performed the best, we tested three different learning rates: 0.05, 0.01, and 0.005. We are working with an imbalanced dataset which requires a smaller learning rate to ensure that the model is not learning too quickly from the larger class (non-popular songs) in the dataset. By setting the learning rate to 0.005, the model is able to learn slowly and understand the patterns in the data without overfitting to the test set. See Figure 8.

4.4 Learning Rate Schedule

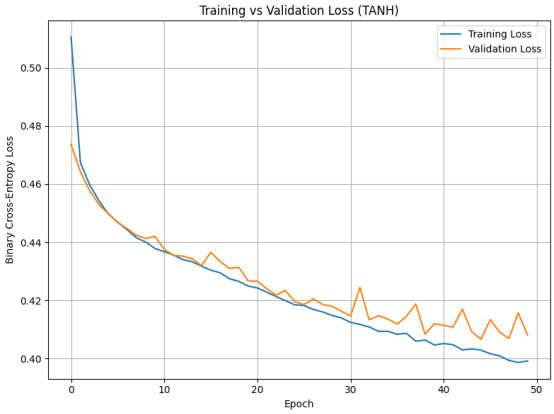
Two different learning rate schedules were tested Exponential Decay and Inverse Time Decay schedule. See Figure 9.



(a) elu



(b) ReLU



(c) tanh

Figure 5: Loss for Different Activation Functions
Though there is mild overfitting, because we are aiming for the best PR-AUC, the little overfitting is expected to reach our goal of determining popular songs.

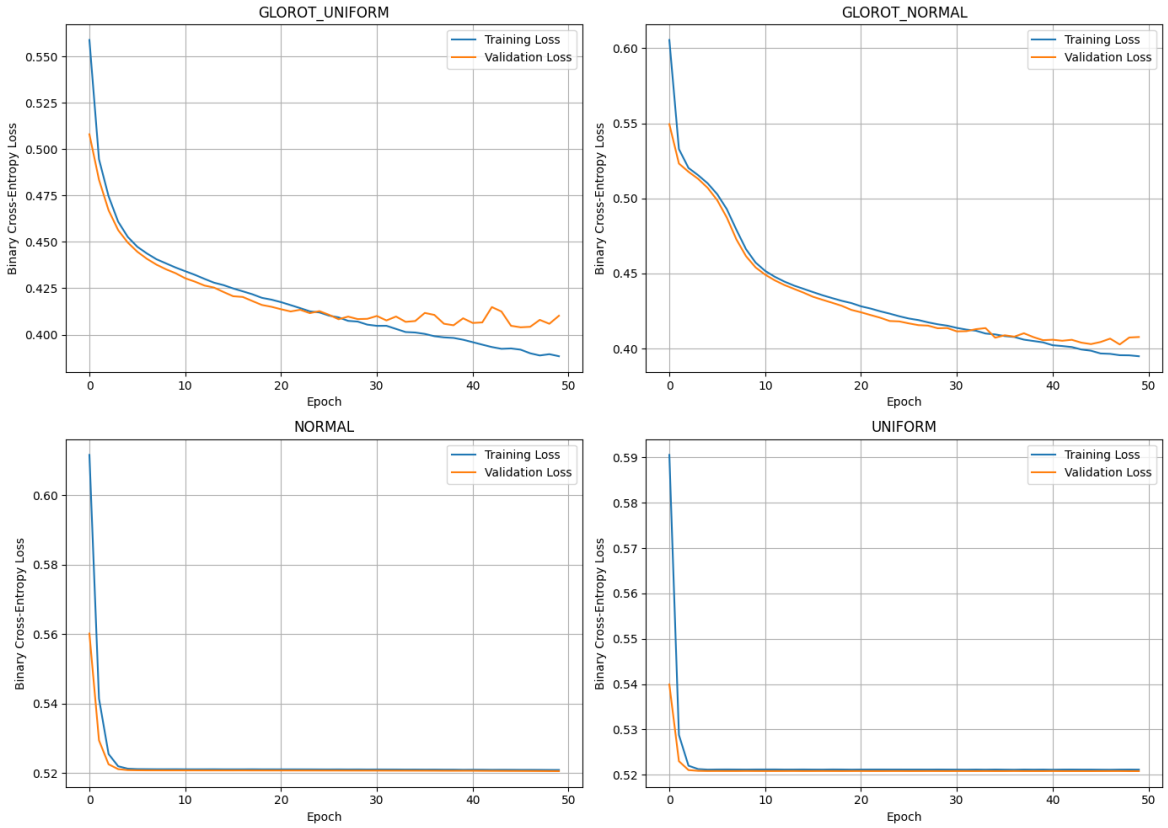
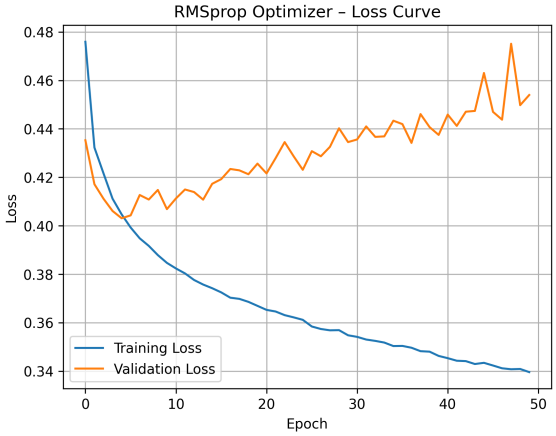
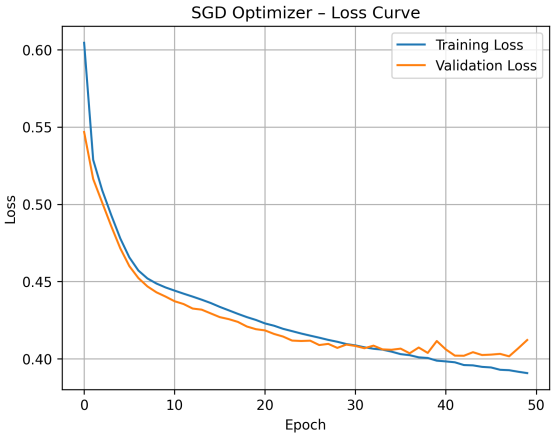


Figure 6: Kernel Initializers Loss Graph Comparisons
Comparing the graphs, glorot-normal performs the best as it does not plateau as quickly as normal and uniform. It also does not overfit as drastically as glorot-uniform.

(a) Adam



(b) RMSprop

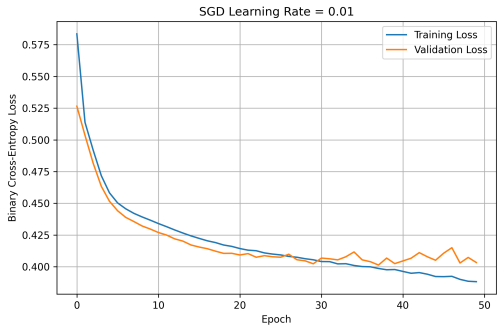


(c) SGD

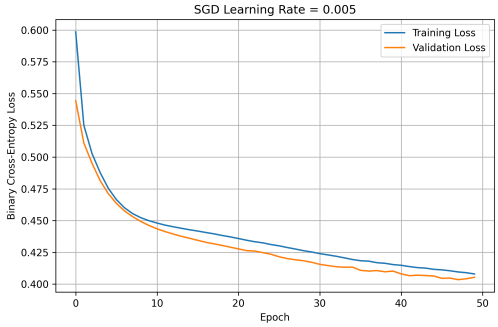
Figure 7: Loss for Different Activation Functions

Both Adam and RMSprop overfit because they learn too quickly and too locally for our problem. This is amplified since our dataset is imbalanced. SGD is able to understand our model better by learning slowly and noisily.

(a) LR = 0.05

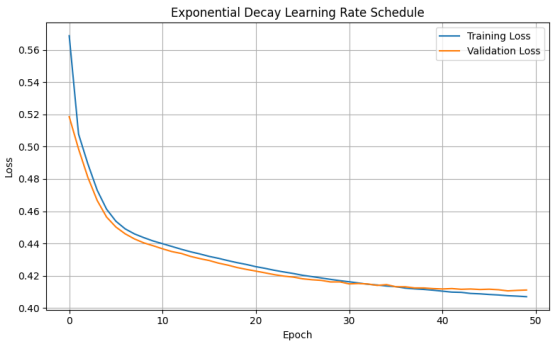


(b) LR = 0.01

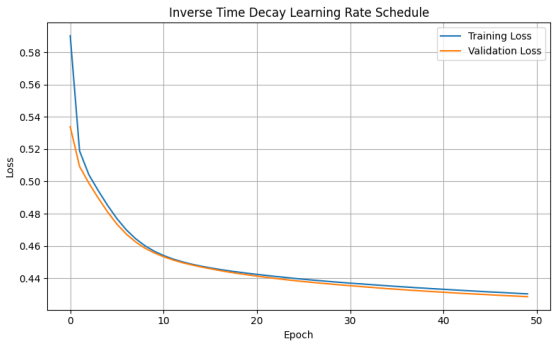


(c) LR = 0.005

Figure 8: Learning Rates
A learning rate of 0.005 performed the best. 0.05 and 0.01 were too large learning rates that caused the model to learn quickly, but overfit becuase of the data imbalance.



(a) Exponential Decay



(b) Inverse Time Decay

Figure 9: Learning Rate Schedule Comparison
Although there is slight overfitting, exponential decay learning rate schedule allows the model to have better accuracy.

5 Hyperparameter Summary

1. Balanced vs Unbalanced: Unbalanced dataset is used to emulate reality. There are more non-popular songs than popular songs. To accurately predict popular songs, the distribution should be similar to reality.
2. PCA vs All Features: Using PCA caused too much abstraction and the model was unable to learn as well, therefore, all features were used.
3. Train-validation-test Split: A 80-10-10 split was used as it provided more training data compared to 70-20-10 to help the model better train and understand patterns.
4. Number of Hidden Layers: 2 hidden layers were used to provide some complexity to the model, but not add too much noise.
5. Hidden Layer Size: 20 was selected as it provided enough complexity for the model without adding too much noise.
6. Dropout: Dropout was 0 as the model did not have enough of the rare class. Dropping would cause the model to lose information.
7. Activation Function: ReLU was selected as it did not cause the model to overfit compared to elu and tanh.
8. Initializer: Glorot-normal performed the best will less overfitting than glorot-uniform and the loss plateaued in normal and uniform initializers.
9. Output Activiation: Sigmoid was used for the output activiation since this is a binary classification problem.
10. Loss Function: Hinge loss provided the best PR-AUC score compared to binary cross entropy though both had the same accuracy.
11. Optimizer: SGD performed the best as the other optimizers learned too quickly causing them to overfit on the common class.
12. Learning Rate Schedule: There was no big difference between the two, but exponential decay allowed the model to have slightly better accuracy.
13. Batch Size: There is a small amount of data, so larger batch sizes were better in performance and efficiency.

6 Results

With this model, we were able to produce the following results:

```
===== FINAL MODEL PERFORMANCE =====
Accuracy:  0.5478
Precision: 0.3109
Recall:    0.9048
F1 Score:  0.4628
ROC-AUC:   0.7252
PR-AUC:    0.3474

Confusion Matrix:
 [310 379]
 [ 18 171]
```

The final loss graph is shown in Figure 10.

These results are representative of the design decisions that we made. Our objective was to have the model focus on finding popular songs. Because our dataset is based on reality and has fewer popular songs compared to non-popular songs, we focused on PR-AUC over accuracy. This led our final model to have a worse accuracy but better recall rate than the baseline model.

If we consider this practically, a recommendation system from Spotify is more likely to recommend a less popular song in the offchance that it is a hit and becomes popular. This ensures that users are streaming music and remain on the application, compared to if a popular song is missed and less streams and revenue are required because of it.

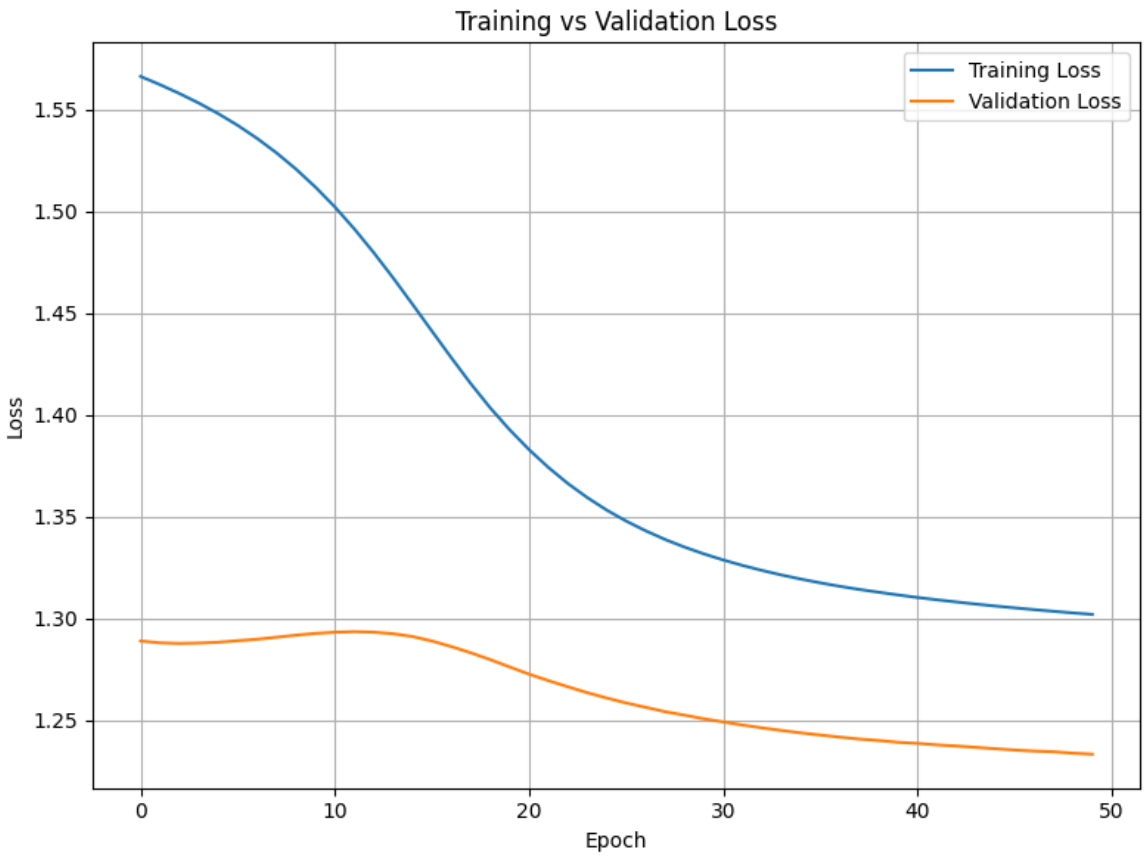


Figure 10: Final MLP Model
The final model has a lower accuracy and a higher loss because there is a focus on ensuring that the model does not miss popular songs. This causes the model to have a higher recall, but lower accuracy and precision.

7 Other Models

Given the performance of the MLP model, we should explore other models to see whether or not they will perform better on our data.

7.1 SVM

Since this is a binary classification between popular and non-popular songs, we can consider using a SVM to classify our data instead. We can use a radial basis function to capture non-linear patterns.

```
===== SVM PERFORMANCE =====
Accuracy:  0.6310
Precision: 0.3555
Recall:    0.8783
F1 Score:  0.5061
ROC-AUC:   0.7923
PR-AUC:    0.4574
Confusion Matrix:
```

```
 [388 301]
 [ 23 166]
```

7.2 Random Forest

This classification problem requires a high accuracy and given the tests that we ran with the MLP model, it is prone to overfitting. Random forest can classify this dataset while reducing overfitting.

```
===== RANDOM FOREST PERFORMANCE =====
Accuracy:  0.7893
Precision: 0.5200
Recall:    0.2751
F1 Score:  0.3599
ROC-AUC:   0.7964
PR-AUC:    0.4683
Confusion Matrix:
 [641  48]
 [137  52]
```

8 Future Work

The final model prioritizes recall and PR-AUC over accuracy to reflect the rarity of popular songs. As a result, accuracy decreases due to an increase in false positives, which is an expected trade-off in imbalanced classification. Loss values exceeding 1 arise from hinge loss penalizing confident margin violations. Future improvements will focus on threshold tuning and calibration to improve accuracy while preserving strong PR-AUC performance.

We found that other models may perform better for this problem and that SVM was able to have a similar confusion matrix while retaining a higher accuracy and PR-AUC. Future work should consider testing different hyperparameters for the model to get better results.